Some of you may already guess that this is simply because our DataLoader is reading the column's `content` that corresponds to the binary image representation and `label_id` corresponding to the original label. We selected one of the most common Computer Vision family models: WideResnet50 from Wide Residual Networks (see the *Arxiv paper* on *Wide Residual Networks* linked in *Further reading* for more information on this family of models). To execute your training, we simply call the training function.

Let's briefly touch on how to move from a single node into the distributed mode. The good news is that you don't have to change much. You simply update your implementation to use a distributed library such as `TorchDistributor`. Here is an example of how you can leverage `TorchDistributor` (please note that if you need to run distributed training for a GenAI task, there is also a similar library for the `DeepSpeed` framework).

```
from pyspark.ml.torch.distributor import TorchDistributor

distributed = TorchDistributor(num_processes=4, local_mode=True, use_gpu=True)
distributed.run(train_distributed, 1, "ddp")
```

Figure 6.29 – Using TorchDistributor to distribute model training

From the code above, you can see that you just need to add a few lines of code to run the same function using a DDP strategy.

Scoring

We perform scoring in the second notebook, `Ch6-02-Scoring Base Model`. It starts with installing all necessary libraries we used for model training in the notebook using pip.

```
!pip install pytorch-lightning==2.1.2 deltalake==0.14.0 deltatorch==0.0.3
evalidate==2.0.2 pillow==10.1.0
dbutils.library.restartPython()
```

Figure 6.30 – Installing notebook scoped for scoring libraries and restarting Python

After installing and loading libraries, we need to find the best model that was trained. For this, we leverage the MLflow SDK, which requires our `experiment_path` name. We are again using utility functions that you can find in the `mlia_utils` folder. Once the best model is found, we check if the GPU cluster is available. If found, we move the model into the GPU.